Towards self-driving codebases

Feb 5, 2026 by Wilson Lin in Research

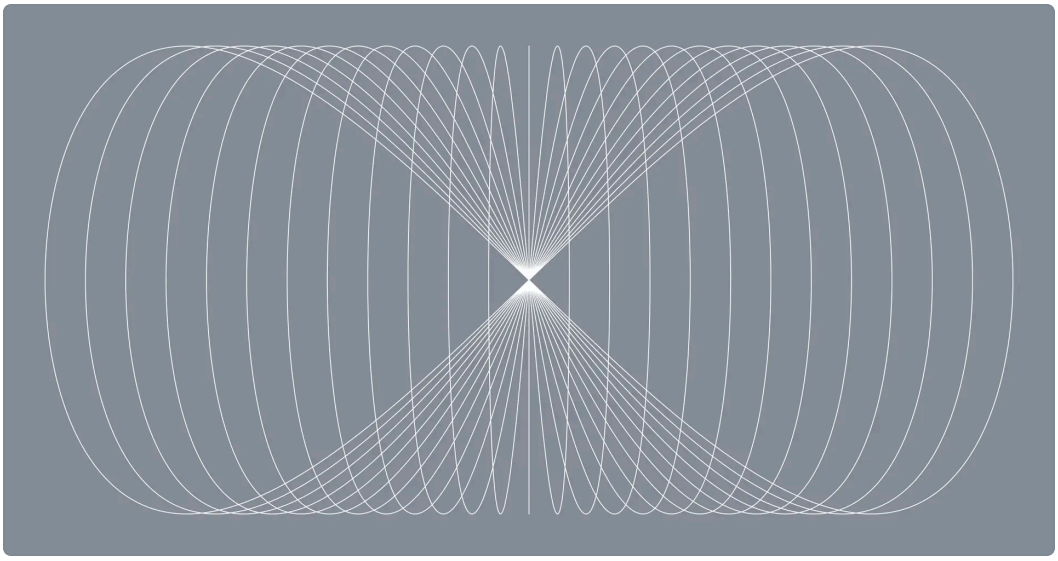


Table of Contents

Background
From single to multi-agent
Self-coordination
Adding structure and roles
Observing and hill-climbing
Continuous executor
Ensuring freshness
Pathological behaviors
The final system design
Removing the integrator
Throughput and tradeoffs
Commit correctness
Synchronization overhead
Infrastructure learnings
Specifying intent to agents
Optimizing prompts
System design learnings
Conclusion

We're excited by the reaction to our research on scaling long-running autonomous coding.

orchestrate many thousands of agents and observe their behavior. By last month, our system was stable enough to run continuously for one week, making the vast majority of the commits to our research project (a web browser). This browser was not intended to be used externally and we expected the code to have imperfections.

However, even with quirks, the fact that thousands of agents could work together to produce work that was almost entirely runnable without human intervention felt like a milestone worth sharing. Since then, we've continued our research, and we wanted to go more in-depth on how the harness was built.

We're also making part of this research available to try for some users.

Background

Our research project started as a personal side project of mine.

A browser felt like an interesting benchmark. It was complex enough to reveal limitations with frontier models, and there are many different subsystems that needed to work together.

My initial plan was to support rendering web pages without JavaScript support. I started by prompting Opus 4.5, asking it to write a detailed plan for building a browser engine. I would repeatedly nudge it to "keep going" to see how far it would go on the plan.

This failed quickly. The model lost track of what it was doing, frequently stopped to proclaim success despite being far from it, and got stuck on complex implementation details. But it showed signs of deep knowledge and intelligence. It could write good code in small pieces.

The core issue was that the browser was too overwhelming of a task and needed to be broken down into subtasks. Next, I had the agent plan out a dependency graph of major work that agents could take up in parallel. Agents were manually spawned for tasks and nudged when they stopped. This increased the throughput, but the results weren't much better. Agents couldn't communicate with each other or provide feedback on the project as a whole. The system needed to be more dynamic.

running agents, so we updated our harness to use OpenAI models based on these experiments.

At this point, the harness could build a simple version of the web browser without JavaScript, but building a complete browser engine with one agent would be prohibitively slow.

This started our next round of research. Could we spend 10x more on compute to get 10x more meaningful throughput?

From single to multi-agent

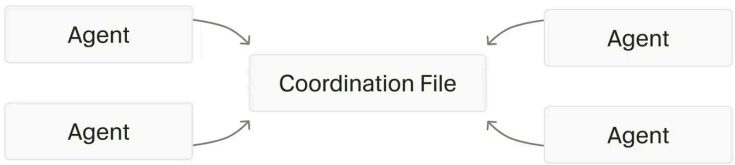
We started a new repository with a simple Rust-based harness.

Rather than dealing with the complexity of distributed systems, we instead ran the harness on a single large Linux VM (Virtual Machine) with lots of resources. To control the harness, we would SSH into the VM and use a simple terminal interface.

We spent more time up front on proper observability into the system. We logged all agent messages, system actions, and command outputs, with timestamps so we could analyze and replay sessions. This was not only helpful for us to manually review, but also for piping back into Cursor to sift through large amounts of data and quickly find patterns.

Self-coordination

Our first multi-agent idea was the most simple: have agents with equal roles use a shared state file to see what others are working on, decide what to work on, and update the file.



We would be the least prescriptive about what to do and instead let the agents figure out how to self-coordinate. This failed quickly.

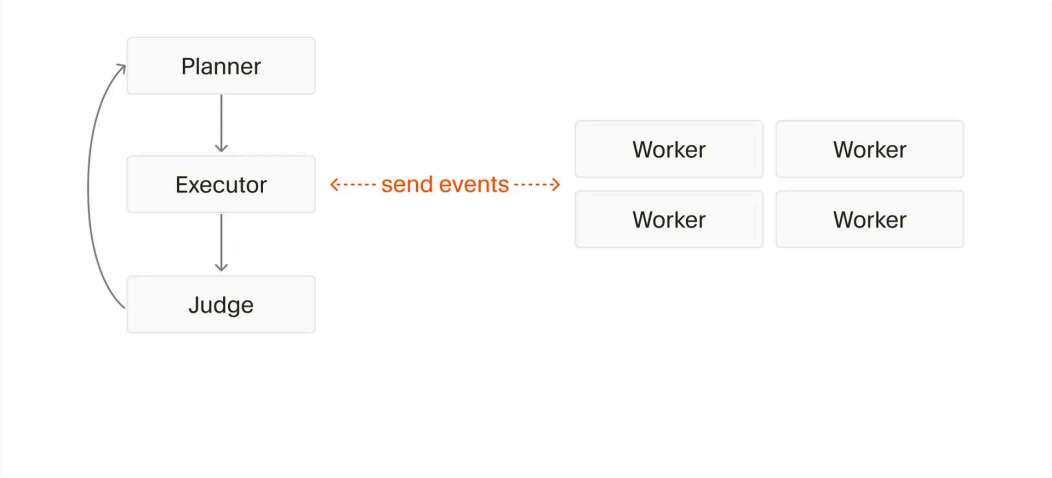
The coordination file quickly created more problems. Agents held locks for too long, forgot to release them, tried to lock or unlock when it was illegal to, and in general didn't understand the significance of holding a lock on the coordination file. Locking is easy to get wrong and narrowly correct, and more prompting didn't help.

Locking also caused too much contention. 20 agents would slow to the throughput of 1-3 with most time spent waiting on locks. We tried giving agents a tool to explicitly wait on another agent's work, but they rarely used it. We also tried a lockless optimistic concurrency control approach, which reduced overhead but didn't eliminate confusion.

The lack of structure between agents meant no single agent took on big, complex tasks. They avoided contention and conflict, opting for smaller and safer changes versus taking responsibility for the project as a whole.

Adding structure and roles

Next, we separated roles to gives the agents ownership and accountability:



A planner would first lay out the exact approach and deliverables to make progress toward the user's instructions. This would be handed to an executor, who became the sole lead agent responsible for ensuring the plan was achieved completely. The executor could spawn tasks for workers, which provided linear scaling and throughput.

For continued movement and accountability, an independent judge ran after the executor finished to determine whether it completed and whether another iteration should run. This resolved many coordination issues. Having a single role dedicated to owning and overseeing execution allowed workers to focus narrowly on their task while the overall system still delivered.

Observing and hill-climbing

Landing on this design required close observation of the system.

If there was a major problem, it would tend to occur repeatedly and across many agents and tool calls. For example, we noticed there was too much contention because many agents were running `git restore` at once. We used Cursor to analyze logs and compare them against our prompts to understand why behavior didn't match expectations.

Ultimately, we found this system to be bottlenecked by the slowest worker. It was too rigid.

Doing all planning upfront also made it hard for the system to dynamically readjust as new issues were discovered. Some agents would end up going

Continuous executor

The next version removed the independent planner.

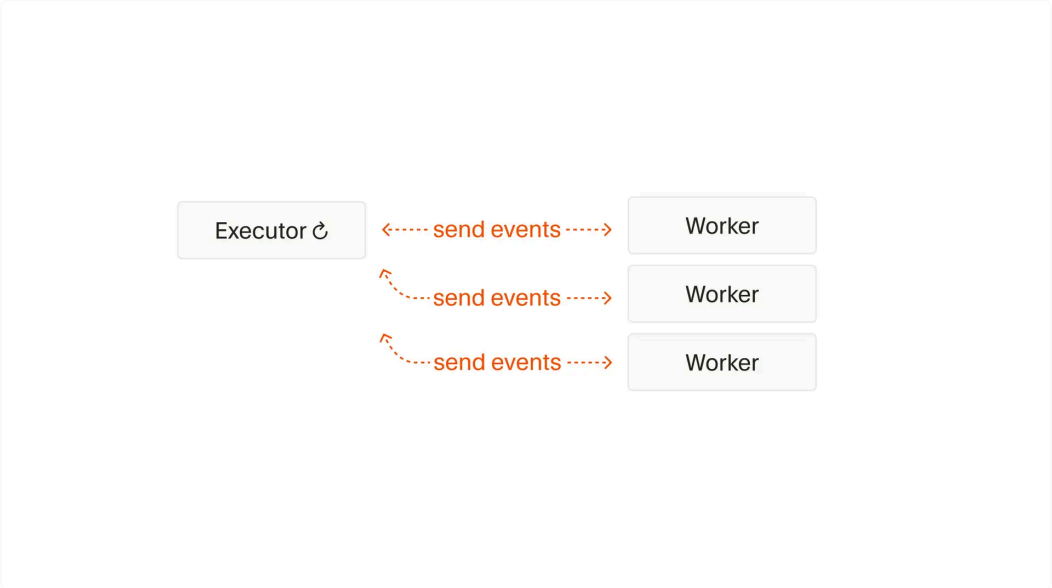
The executor could now also plan how to deliver the goal in addition to spawning tasks. Since it was the sole agent, it didn't need to write a plan anywhere, stick to one static unchanging plan, or rigidly wait for all workers.

Ensuring freshness

To ensure agents across all roles wouldn't drift over long periods of time, we introduced freshness mechanisms:

1. A `scratchpad.md` should be frequently rewritten versus being appended to.
2. Individual agents should automatically summarize when reaching context limits.
3. We added self-reflection and alignment reminders to the system prompts.
4. Agents were encouraged to pivot and challenge assumptions at any time.

The system was now highly dynamic and flexible: it could proactively explore code, reconsider decisions, manage workers, interleave tasks, and continuously reflect the latest information. We found agents were reasonably good at following instructions to completion, so the judge was removed to keep the system simple.



pathological behaviors. It would sleep randomly, stop running agents, do work itself, refuse to plan and spawn more than a few narrowly focused tasks, not properly merge worker changes, and claim premature completion.

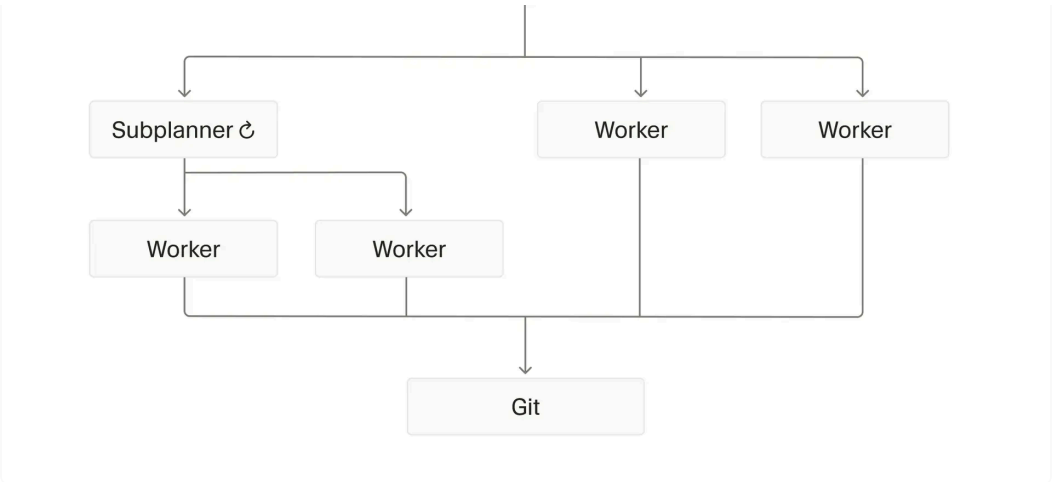
We found it was being given too many roles and objectives simultaneously, including: plan, explore, research, spawn tasks, check on workers, review code, perform edits, merge outputs, and judge if the loop is done. In retrospect, it makes sense it was overwhelmed.

The final system design

The final design incorporates all of our learnings:

1. A root planner owns the entire scope of the user's instructions. It's responsible for understanding the current state and delivering specific, targeted tasks that would progress toward the goal. It does no coding itself. It's not aware of whether its tasks are being picked up or by whom.
2. When a planner feels its scope can be subdivided, it spawns subplanners that fully own the delegated narrow slice, taking full ownership in a similar way but only for that slice. This is recursive.
3. Workers pick up tasks and are solely responsible for driving them to completion. They're unaware of the larger system. They don't communicate with any other planners or workers. They work on their own copy of the repo, and when done, they write up a single handoff that the system submits to the planner that requested the task.

Interestingly, this does represent how some software teams operate today.



Subplanners increase throughput by rapidly fanning out workers while ensuring the whole system remains fully owned and responsible by an agent. This also helped with large projects and tasks where a single planner would otherwise get overwhelmed and develop tunnel vision.

The handoff contains not just what was done, but important notes, concerns, deviations, findings, thoughts, and feedback. The planner receives this as a follow-up message. This keeps the system in continuous motion: even if a planner is "done," it continues to receive updates, pulls in the latest repo, and can continue to plan and make subsequent decisions.

All agents have this mechanism, which allows the system to remain incredibly dynamic and self-converging, propagating information up the chain to owners with increasingly global views, without the overhead of global synchronization or cross-talk.

Removing the integrator

We originally added an integrator for central globally-aware quality control and to remove contention from too many workers trying to push, rebase, resolve conflicts, and merge simultaneously.

It quickly became an obvious bottleneck. There were hundreds of workers and one gate (i.e. "red tape") that all work must pass through. We tried prompt changes, but ultimately decided it was unnecessarily and could be removed to simplify the system.

period of one week. Once the system started, it didn't require any intervention from us.

There were intentional tradeoffs to achieve this throughput.

Commit correctness

When we required 100% correctness before every single commit, it caused major serialization and slowdowns of effective throughput. Even a single small error, like an API change or typo, would cause the whole system to grind to a halt. Workers would go outside their scope and start fixing irrelevant things. Many agents would pile on and trample each other trying to fix the same issue.

This behavior wasn't helpful or necessary. Allowing some slack means agents can trust that other issues will get fixed by fellow agents soon, which is true since the system has effective ownership and delegation over the whole codebase. Errors arise then get fixed quickly. The error rate remains small and constant, perhaps rarely completely clean but steady and manageable, not exploding or deteriorating.

and does a quick fixup pass before release.

Synchronization overhead

Sometimes multiple agents touch the same file or refactor the same code. Instead of trying to stamp these out completely or overengineer a solution, we accept some moments of turbulence and let the system naturally converge and settle over a short period of time.

This spends some extra tokens and creates local contention, but keeps the system overall simpler: easier to align models and not overwhelm them, easier to manage and observe, less friction, and better global productivity. It also avoids overly complex approaches.

Infrastructure learnings

Each multi-agent run ran on its own large machine with ample system resources, to avoid premature complexity around distributed systems. This was a good fit, as most runs would peak at several hundred agents, which typically saturated but did not overprescribe these machines. This architecture made it easier to observe system metrics, and share and copy state where necessary.

After limiting RAM usage of agents, the disk became the hotspot. Especially with a monolith project, hundreds of agents compiling simultaneously would result in many GB/s reads and writes of build artifacts. This had a significant impact on the overall throughput of the harness, which was an interesting lesson: the project structure, architectural decisions, and developer experience can affect token and commit throughput, simply because working with the codebase (e.g. compilation) dominates time, instead of ideally thinking and coding.

There were also constraints and inefficiencies in the general development environment: things that make sense or aren't significant for a single user workspace, can stick out with hundreds of agents doing the same thing on one machine. One trivial way to solve this is to give each agent its own machine. But there are interesting low-hanging opportunities for large efficiency gains just by rethinking and redesigning some of these primitives and tools.

mechanisms from concurrent systems like databases make these work just as well in multi-agent systems? All agents have their own copy of the repo, but most files and artifacts are identical; could adding simple copy-on-write and deduplication features, found in more sophisticated production storage systems, bring similar easy wins to a typically "single-user" system without building separate infrastructure?

Specifying intent to agents

Instructions given to this multi-agent system were very important.

Initially, we didn't make them our primary goal, but instead aimed for a stable and effective harness. But the significance of instructions became apparent quickly. We were essentially interacting with a typical coding agent, except with orders of magnitude more time and compute. This amplifies everything, including suboptimal and unclear instructions.

Spending more time on the initial instructions makes sense. Ultimately, agents are still agents: trained to follow your instructions strictly, go down those paths, not change or override them, even if they're bad.

We wanted to see success in our research projects, so we altered our initial instructions as the project and harness evolved. We were learning how to build a browser alongside learning how to operate this new multi-agent system, and could see poor or underspecified specifications reflected in the quality of the outputs, which was not due to the harness itself. The harness was merely following our instructions exactly.

Some examples from the browser project:

- Initially, the instructions focused on implementing specs and squashing bugs. Instructions like "spec implementation" were vague enough that agents would go deep into obscure, rarely used features rather than intelligently prioritizing.
- We assumed implicitly that there were performance expectations within user-friendly bounds. But it took explicit instructions and enforced timeouts to force agents to balance performance alongside other goals.
- For complex parts of the system, agents may write code that has memory leaks or causes deadlocks. Humans would notice this, but it wasn't always obvious to agents. Explicit process-based resource management

Our first version of the simple browser without JavaScript converged on an architecture that was unfit to evolve into a full browser. This was a failure of the initial specification.

Similarly, while the agents were told the project was a browser from scratch, they still pulled some dependencies they could've implemented themselves, or used as temporary scaffolding while proper implementation was underway. This was oversight in instructions. A later run explicitly laid out the dependency philosophy and which libraries must not be used, which corrected this.

That later run also did a major restructuring into many self-contained crates, moving away from a monolith. The repo was in a heavily broken state, but the multi-agent system converged toward working code in a few days. This showed the system has strong capability to work collaboratively and intelligently, holding across totally broken states instead of degrading further or getting stuck. This run also spent far less time waiting on compilation, running at multiple times more throughput than previously.

Architecture and instructions matter. Agents have immense engineering skill but will follow instructions to the end, good or bad. Finding the balance between overly narrow metrics and unstructured freedom was tricky, as was knowing what was obvious versus what needed explicit mention.

All of this indicates the importance of eliciting, specifying, and understanding intent, which becomes even more significant at this scale. Steerability and observability will be interesting research areas to continue exploring.

Optimizing prompts

Prompting was a significant part of the evolution process.

We found it was better to not instruct for things the model knows how to do, only things it doesn't know (e.g. multi-agent collaboration) or that are specific to the relevant domain (e.g. how to run tests, your deploy pipeline). Treat the model like a brilliant new hire who knows engineering but not your specific codebase and processes.

Models generally do good things by default. Constraints are defining their boundaries.

Avoid checkbox mentality for higher-level or deeper tasks. Give detailed instructions about your intent, but remember giving specific things to do tends to make the model focus on achieving those rather than the wider scope. You also implicitly deprioritize unlisted things. Typically, it's better to let the model use its judgment and agency.

We did find it useful to give concrete numbers and ranges when discussing quantity of scope. Instructions like "generate many tasks" tend to produce a small amount: conservative default, playing it safe, technically still following instructions. "Generate 20-100 tasks" conveys the intent is larger scope, it should be ambitious, and we observed very different wider behavior.

System design learnings

We established some principles from our research:

1. **The system should be anti-fragile.** As we scale the number of agents running simultaneously, we also increase the probability of failure. Our system needs to withstand individual agents failing, allowing others to recover or try alternative approaches.
2. **Empirical over assumption-driven.** We wanted to use data and observation to make adjustments, rather than coming in with assumptions about how it should work based on human organizations or existing system designs.
3. **Design for throughput explicitly.** This meant trading off other aspects of coding, like accepting a small but stable rate of errors that requires a final reconciliation pass, instead of perfectly working code 100% of the time that would dramatically slow down the system.

Filed under: Research
Author: Wilson Lin

These systems tend to be elegantly simple when done right, but it wasn't clear *which* simple approach would work until we explored many different approaches. The current system design has been running with minimal overhead and provides linear scaling of token throughput in a useful manner. No major further iterations have been necessary on the harness.

Conclusion

← Previous post
Securely indexing large codebases

Next post →
NVIDIA commits 3x more code across 30,000 developers with Cursor

operate today. These models were not explicitly trained in this way, which suggests it's emergent behavior and possibly the correct way of structuring software projects after all.

We will be continuing to research extremely long-running agents, with our findings informing the future of our product. ■

© 2026 Anysphere, Inc. SOC 2 Certified